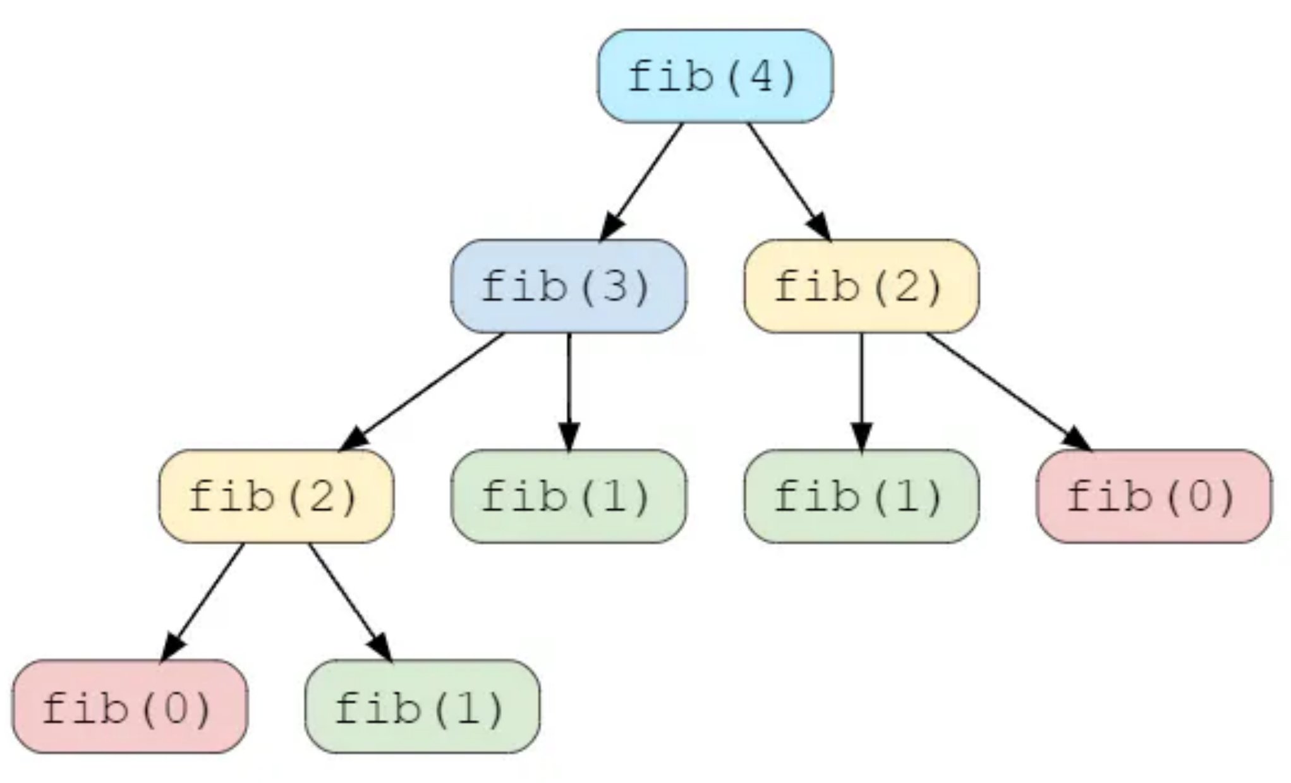
Introduction

- What is Dynamic Programming?

Pattern 1: 0/1 Knapsack

- Introduction
- 0/1 Knapsack
- Solution: 0/1 Knapsack
- Equal Subset Sum Partition
- Solution: Equal Subset Sum Partition
- Subset Sum
- Solution: Subset Sum
- Minimum Subset Sum Difference
- Solution: Minimum Subset Sum Difference
- Count of Subset Sum
- Solution: Count of Subset Sum
- Target Sum



Recursion tree for calculating Fibonacci numbers

We can clearly see the overlapping subproblem pattern here, as `fib(2)` has been evaluated twice and `fib(1)` has been evaluated three times.

Optimal Substructure Property

Any problem has optimal substructure property if its overall optimal solution can be constructed from the optimal solutions of its subproblems. For Fibonacci numbers, as we know,

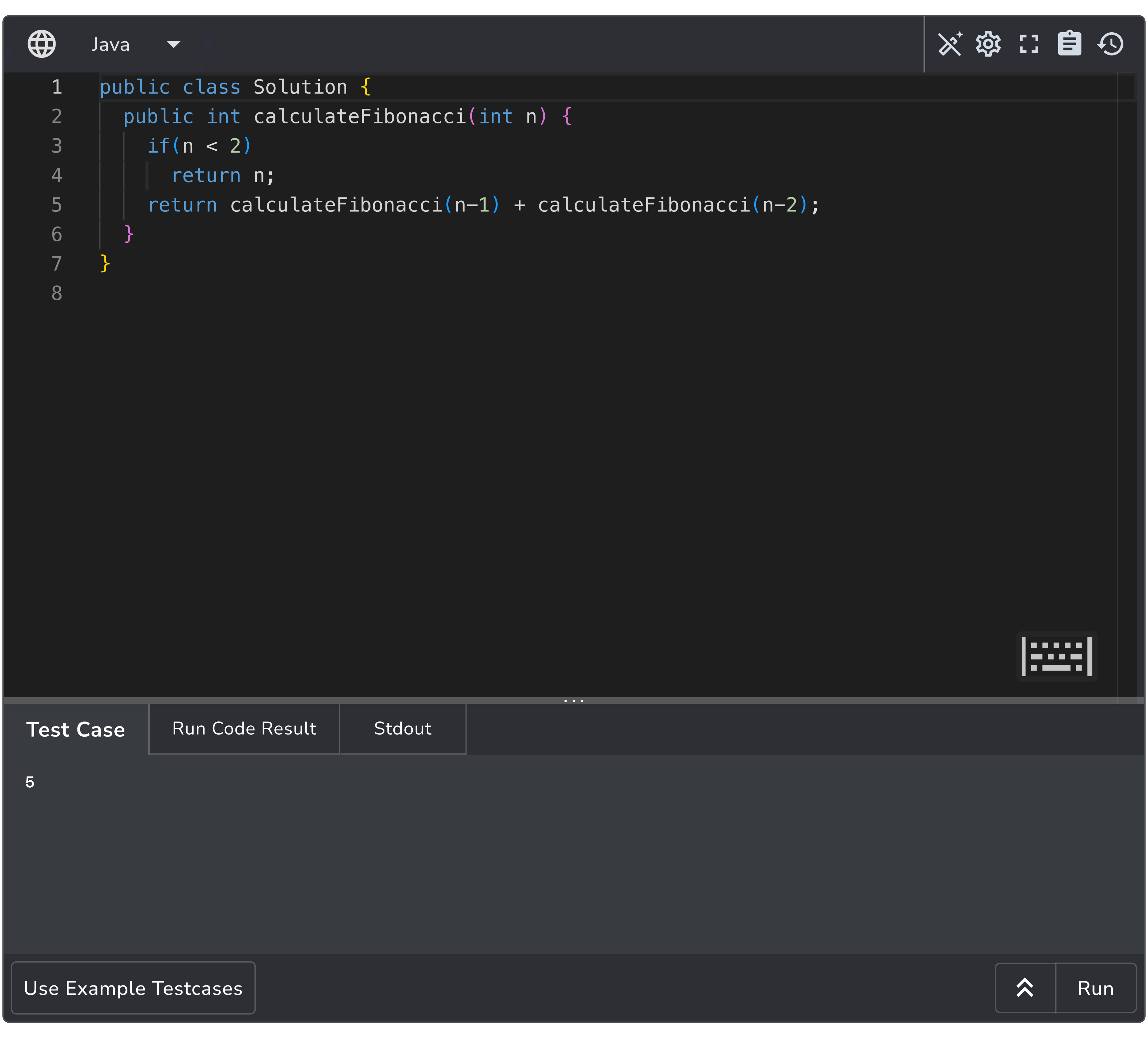
```
Fib(n) = Fib(n-1) + Fib(n-2)
```

This clearly shows that a problem of size 'n' has been reduced to subproblems of size 'n-1' and 'n-2'. Therefore, Fibonacci numbers have optimal substructure property.

Top-down with Memoization

In this approach, we try to solve the bigger problem by recursively finding the solution to smaller sub-problems. Whenever we solve a sub-problem, we cache its result so that we don't end up solving it repeatedly if it's called multiple times. Instead, we can just return the saved result. This technique of storing the results of already solved subproblems is called **Memoization**.

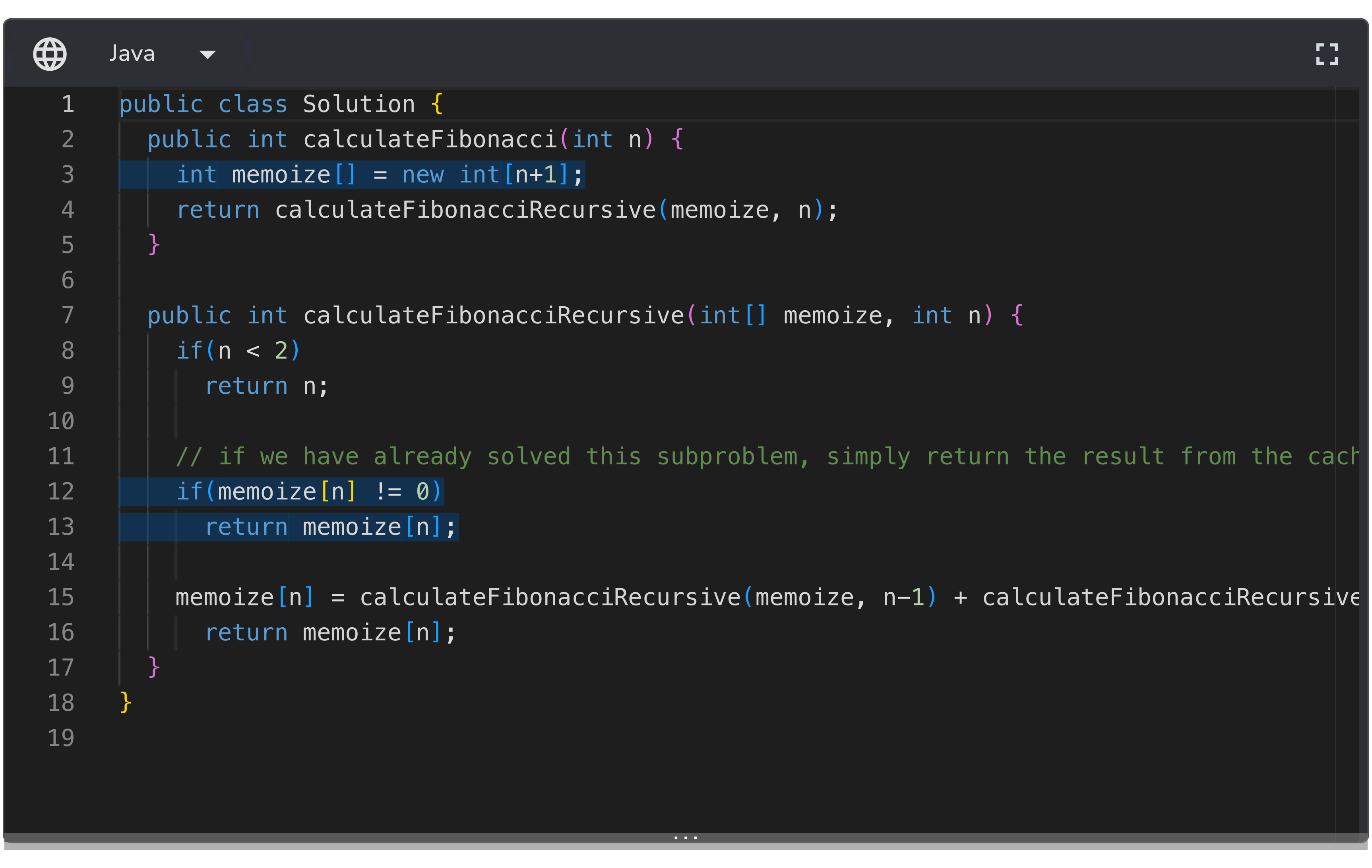
We'll see this technique in our example of Fibonacci numbers. First, let's see the non-DP recursive solution for finding the nth Fibonacci number:



```java
public class Solution {
  public int calculateFibonacci(int n) {
    if(n < 2)
      return n;
    return calculateFibonacci(n-1) + calculateFibonacci(n-2);
  }
}
```

5

As we saw above, this problem shows the overlapping subproblems pattern, so let's make use of memoization here. We can use an array to store the already solved subproblems (see the changes in the highlighted lines).



```java
public class Solution {
  public int calculateFibonacci(int n) {
    int memoize[] = new int[n+1];
    return calculateFibonacciRecursive(memoize, n);
  }

  public int calculateFibonacciRecursive(int[] memoize, int n) {
    if(n < 2)
      return n;

    // if we have already solved this subproblem, simply return the result from the cach
    if(memoize[n] != 0)
      return memoize[n];

    memoize[n] = calculateFibonacciRecursive(memoize, n-1) + calculateFibonacciRecursive
      return memoize[n];
  }
}
```

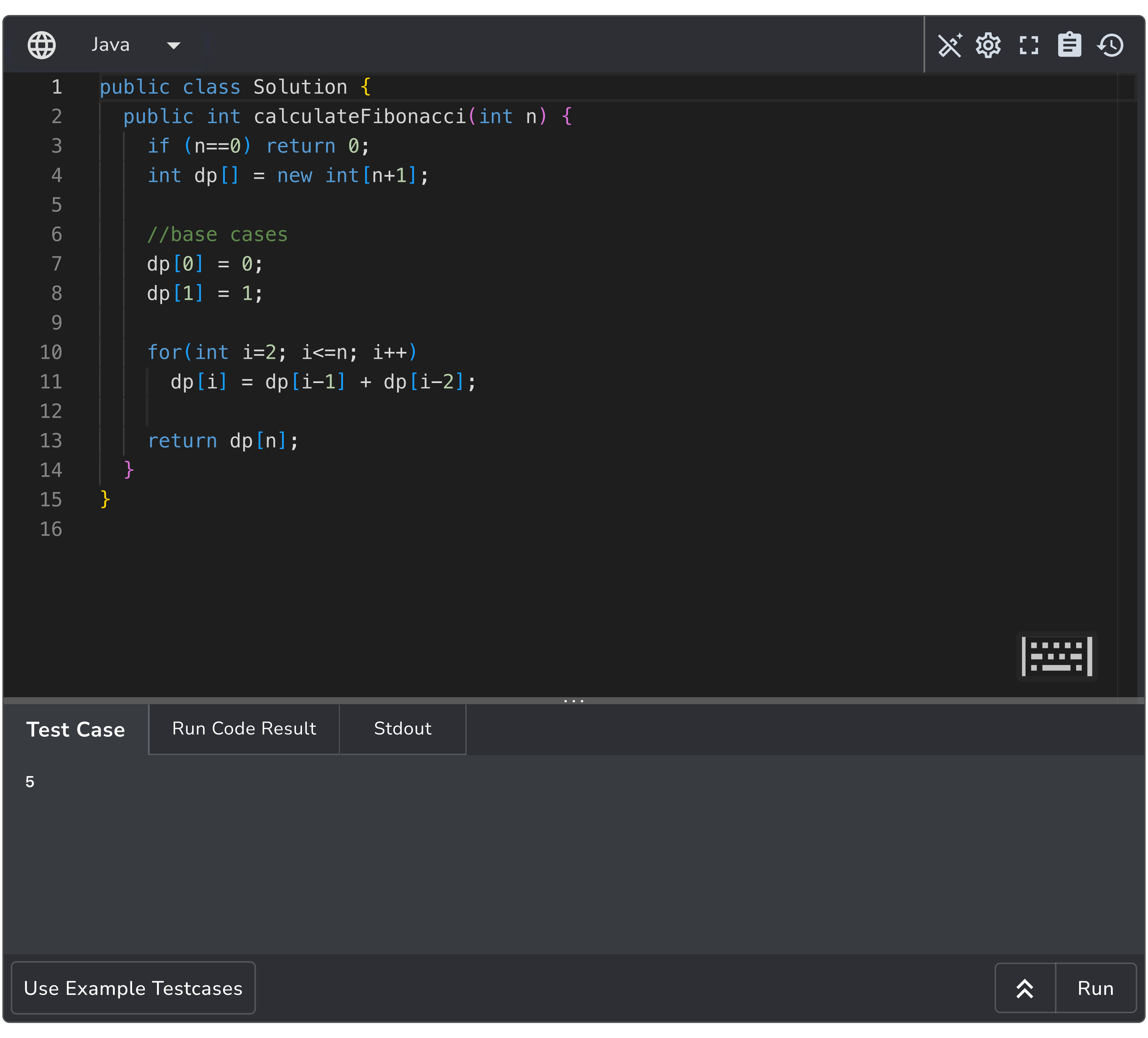
Bottom-up with Tabulation

Tabulation is the opposite of the top-down approach and avoids recursion. In this approach, we solve the problem "bottom-up" (i.e. by solving all the related sub-problems first). This is typically done by filling up an n-dimensional table. Based on the results in the table, the solution to the top/original problem is then computed.

Tabulation is the opposite of Memoization, as in Memoization we solve the problem and maintain a map of already solved sub-problems. In other words, in memoization, we do it top-down in the sense that we solve the top problem first (which typically recurses down to solve the sub-problems).

Let's apply Tabulation to our example of Fibonacci numbers. Since we know that every Fibonacci number is the sum of the two preceding numbers, we can use this fact to populate our table.

Here is the code for our bottom-up dynamic programming approach:



```java
public class Solution {
  public int calculateFibonacci(int n) {
    if (n==0) return 0;
    int dp[] = new int[n+1];

    //base cases
    dp[0] = 0;
    dp[1] = 1;

    for(int i=2; i<=n; i++)
      dp[i] = dp[i-1] + dp[i-2];

    return dp[n];
  }
}
```

5

In this course, we will always start with a brute-force recursive solution, which is the best way to start solving any DP problem! Once we have a recursive solution then we will apply Memoization and Tabulation techniques.

Let's apply this knowledge to solve some of the frequently asked DP problems.

Next →
Introduction

☐ Mark as Completed